A Dependency Manager for PHP

Latest: **2.10.2** (changelog)

Getting Started | Download

Documentation | Browse Packages

Issues | GitHub

Authors: Nils Adermann, Jordi Boggiano and many community contributions

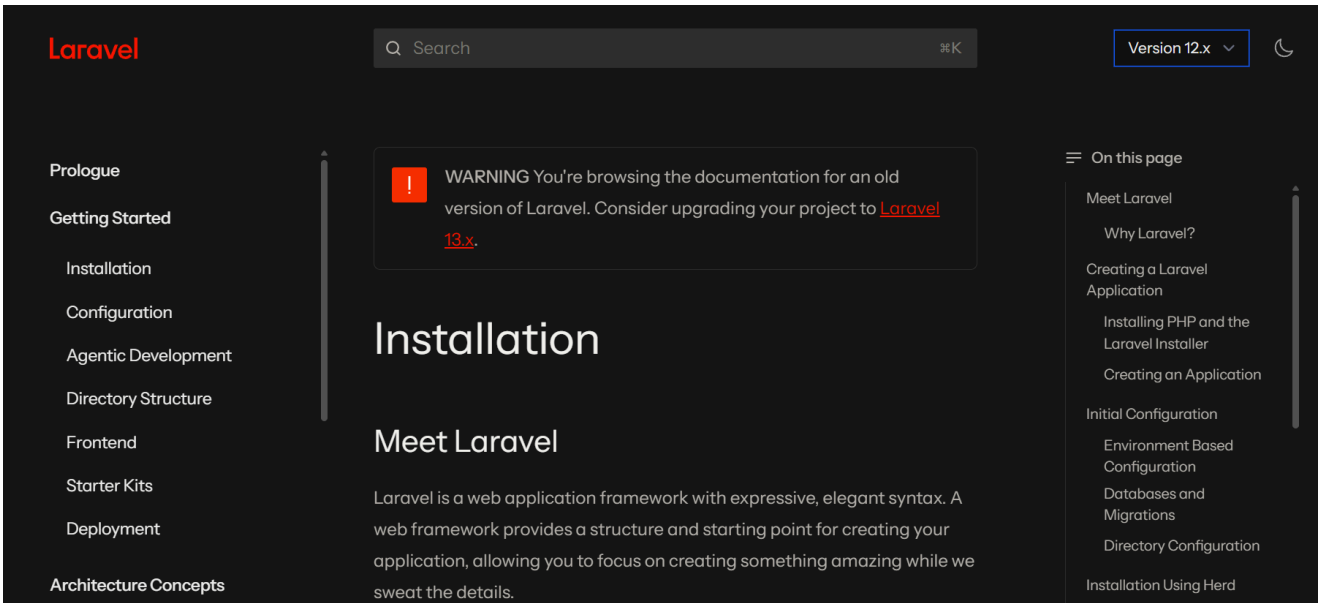
Commercial support & consulting available through:

PRIVATE PACKAGIST

Sponsor Composer & Packagist.org

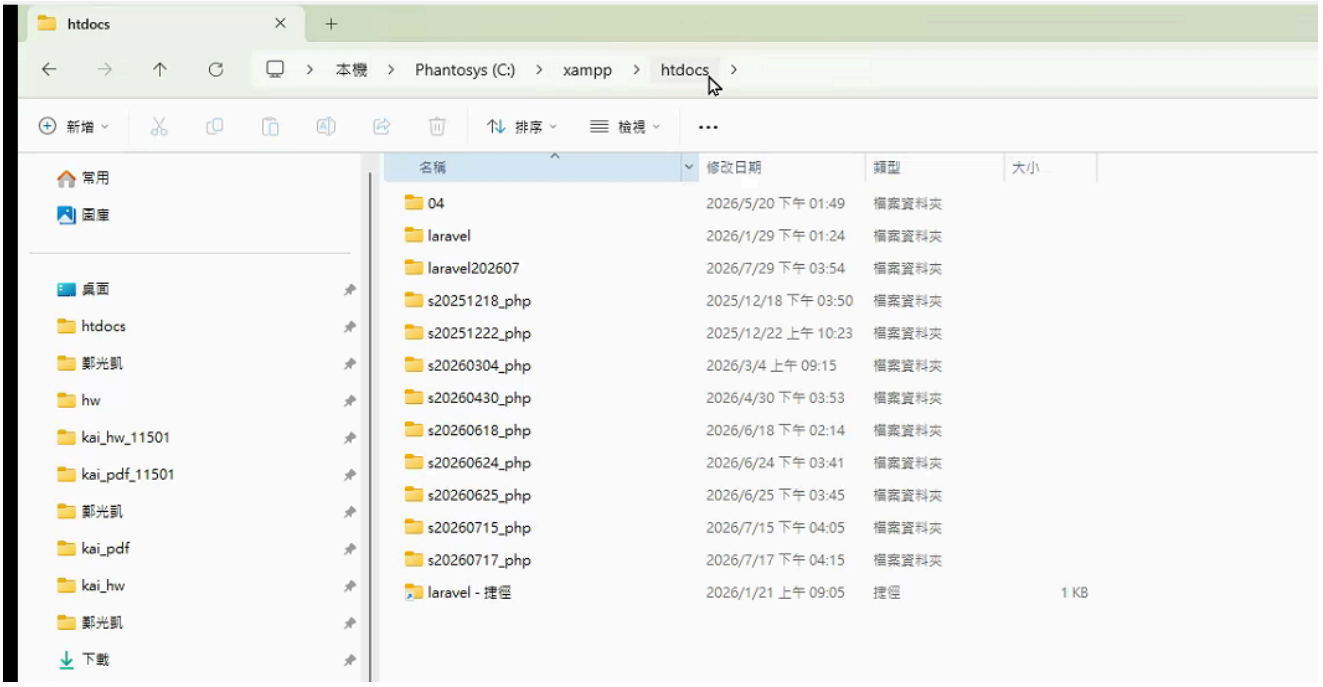
Logo by: Max Grigorian

https://laravel.com/

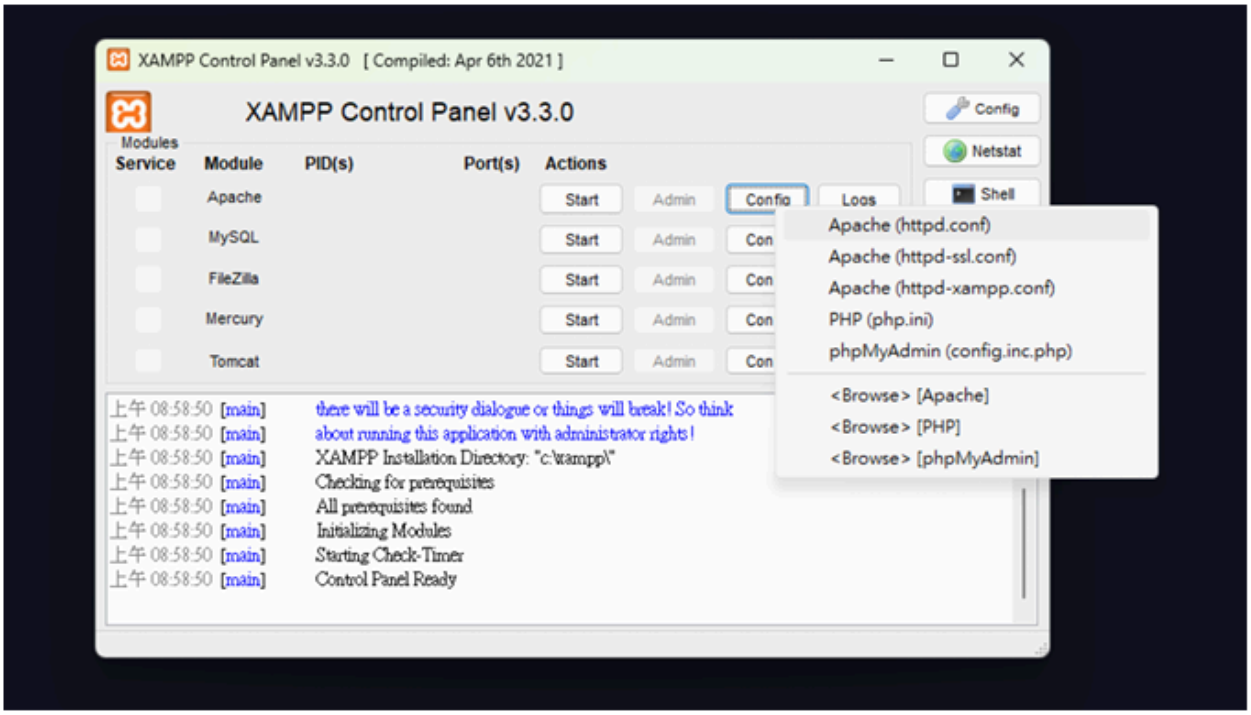



```
User@1113-Teacher MINGW64 /c/xampp/htdocs/laravel202607
$ composer create-project "laravel/laravel:^12.0" s2026073001
Creating a "laravel/laravel:^12.0" project at "./s2026073001"
Installing laravel/laravel (v12.12.2)
  - Installing laravel/laravel (v12.12.2): Extracting archive
Created project in C:\xampp\htdocs\laravel202607\s2026073001
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 111 installs, 0 updates, 0 removals
  - Locking brick/math (0.14.8)
  - Locking carbonphp/carbon-doctrine-types (3.2.0)
  - Locking dflydev/dot-access-data (v3.0.3)
  - Locking doctrine/inflector (2.1.0)
  - Locking doctrine/lexer (3.0.1)
  - Locking dragonmantank/cron-expression (v3.6.0)
```

```
  INFO  Preparing database.

  Creating migration table ..........................................

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ..............................
  0001_01_01_000001_create_cache_table ..............................
  0001_01_01_000002_create_jobs_table ...............................


User@1113-Teacher MINGW64 /c/xampp/htdocs/laravel202607
$ ls
s2026072901-init/  s2026072901-init.7z  s2026073001/

User@1113-Teacher MINGW64 /c/xampp/htdocs/laravel202607
$ cd s2026073001/

User@1113-Teacher MINGW64 /c/xampp/htdocs/laravel202607/s2026073001
```

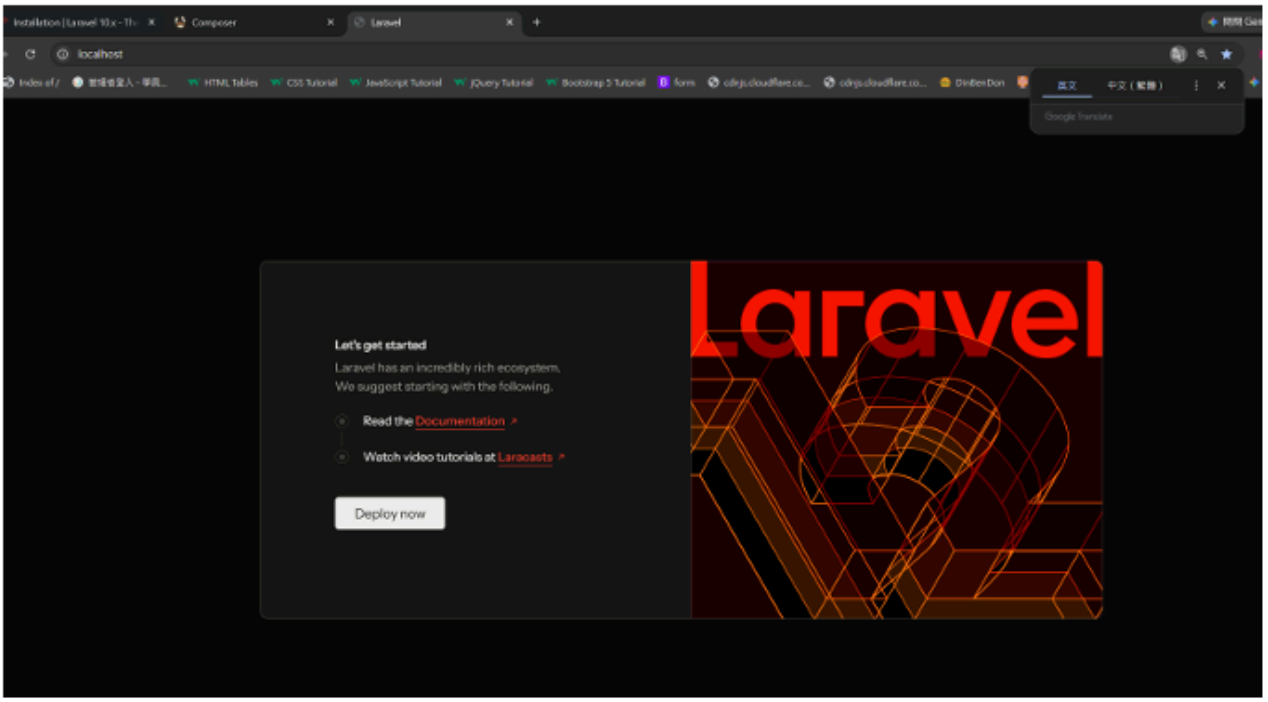


php artisan make:controller ProvisionServer --invokable



DocumentRoot "C:/xampp/htdocs/laravel202607/s2026073001/public"

<Directory "C:/xampp/htdocs/laravel202607/s2026073001/public">



2.route

Refreshing on Save

When your application is built using traditional server-side rendering with Blade, Vite can improve your development workflow by automatically refreshing the browser when you make changes to view files in your application. To get started, you can simply specify the `refresh` option as `true`.

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
    plugins: [
        laravel({
            // ...
            refresh: true,
        }),
    ],
});
```

When the `refresh` option is `true`, saving files in the following directories will trigger the browser to perform a full page refresh while you are running `npm run dev`:

3.controller

Controllers

Introduction

Instead of defining all of your request handling logic as closures in your route files, you may wish to organize this behavior using "controller" classes. Controllers can group related request handling logic into a single class. For example, a `UserController` class might handle all incoming requests related to users, including showing, creating, updating, and deleting users. By default, controllers are stored in the `app/Http/Controllers` directory.

Writing Controllers

Basic Controllers

To quickly generate a new controller, you may run the `make:controller` Artisan command. By default, all of the controllers for your application are stored in the `app/Http/Controllers` directory:

```
1    php artisan make:controller UserController
```

4.view

Views

Introduction

Of course, it's not practical to return entire HTML documents strings directly from your routes and controllers. Thankfully, views provide a convenient way to place all of our HTML in separate files.

Views separate your controller / application logic from your presentation logic and are stored in the `resources/views` directory. When using Laravel, view templates are usually written using the Blade templating language. A simple view might look something like this:

```
<!-- View stored in resources/views/greeting.blade.php -->

<html>
    <body>
        <h1>Hello, {{ $name }}</h1>
    </body>
</html>
```

views

設計原理

1. Convention over Configuration 慣例優於設定
2. Don't Repeat Yourself 不要重複造輪子

懶人包 希望 有一天 你可以自己獨立做出來

Laravel 預設首頁

那我們開啟 `routes/web.php` 檔案中，可以看到下面這一段預設的路由

```
Route::get('/', function () {
    return view('welcome');
});
```

這段的意思就是代表，如果我們網址沒有輸入其他的目錄，就會顯示 welcome 這個網頁，那 welcome 這個顯示的畫面，就放在 `resources/view` 底下名為 welcome.blade.php 的檔案中，打開來看就是簡單的 HTML。

那現在我們了解路由的含義，現在換我們來實作一個看看，首先我們希望網址列輸入 `https://127.0.0.1:8000/hello-world` 可以出現 `hello world` 的字樣，那我們可以到 web.php 中，新增下面這個路由

```
Route::get('/hello-world', function () {
    return view('hello-world');
});
```